# KACHUNO

Small steps. Steady progress.

**CONSISTENCY**
PATIENT BUILDING
MEANINGFUL PROGRESS

## 7-DAY PROJECT BLUEPRINT

A practical build plan for a responsive productivity app, cloud-backed web release, and Android packaging.

**Prepared for:** Antigravity IDE implementation workflow
**Stack:** React · TypeScript · Vite · Supabase · Capacitor · GitHub
**Approach:** Build one approved day at a time; test, document, then commit.

Version 1.0 · 21 October 2026

Planning target, not a promise of Google Play approval or production readiness in seven calendar days.

# How to use this blueprint

This document is the working specification for Kachuno Version 1. Upload it to Antigravity and use the master prompt near the end. Do not ask the agent to build all seven days at once. Approve one day, review the diff and test results, then explicitly approve the next day.

**Core rule:** preserve a working application over chasing a schedule. If a day is not complete, document the blocker and carry the unfinished work forward rather than marking it done.

## Document map

| Section | Purpose |
|---|---|
| Product vision and scope | What Kachuno is, what Version 1 includes, and what is intentionally excluded. |
| Architecture and data model | Technology choices, app structure, database fields, and security boundaries. |
| Seven-day plan | Objectives, implementation steps, likely files, testing, completion criteria, commits, and troubleshooting. |
| GitHub workflow | Safe daily Git routine and repository hygiene. |
| Antigravity master prompt | Copy-ready instructions for controlled, verifiable implementation. |
| Release and acceptance | Deployment, Android, Google Play, and final verification checklists. |
| Daily log template | Repeatable record of work, tests, commits, blockers, and next steps. |

## Assumptions and constraints

- Budget target: ■0 for development tools and hosting where free tiers suffice. Free-tier limits, eligibility, and policies can change; check them before relying on them.
- The developer is learning. Prefer clear, maintainable code and explain unfamiliar decisions briefly.
- Repository state is unknown. Antigravity must inspect it first and adapt instead of replacing files blindly.
- Seven days means seven focused work sessions, not guaranteed calendar days or guaranteed store approval.
- Use only test data while validating privacy and authentication. Never place real credentials or private user data in source control.

# 1. Product vision and scope

## Vision

Kachuno helps people make steady progress through manageable daily actions. Its turtle-inspired identity should communicate patience, reliability, and calm focus—not childishness or pressure. The product should tell the truth about progress and make the next useful action easy to see.

## Product principles

- **Honest by default:** every statistic must have a defined formula and time period. No fabricated streaks, fake progress, or guilt-based copy.
- **Low friction:** common task actions should be obvious and quick. Keep the primary navigation consistent across screen sizes.
- **Private by design:** each signed-in user can access only their own records, enforced by PostgreSQL Row Level Security (RLS), not merely hidden in the UI.
- **Accessible:** semantic HTML, visible keyboard focus, readable contrast, labelled controls, and understandable errors.

- **Resilient:** handle loading, empty, offline/network failure, invalid input, expired sessions, and database errors.
- **Small dependencies:** avoid adding libraries unless they solve a clear problem.

## Version 1 feature scope

| Area | Included in Version 1 | Definition of done |
|---|---|---|
| My Day | Due-today tasks, overdue tasks, completion counts, focus time for today. | Counts and lists derive from saved records; no hard-coded demo metrics. |
| Tasks | Create, edit, complete, reopen, delete; description; due date; priority; category; filters. | All CRUD operations work, validate input, and persist. |
| Calendar | Monthly and weekly views, tasks on due dates, date selection. | Date placement is correct across month boundaries and local timezone. |
| Focus | Start, pause, resume, reset; optionally link task; save completed sessions. | Elapsed time is not lost when a tab redraws; completed session duration is accurate. |
| Progress | History, completion rate, overdue count, focus minutes, weekly trends. | Formulas and periods are explicit; zero-data periods are handled safely. |
| Authentication | Sign up, sign in, sign out. | User can recover from errors and session state is handled. |
| Cloud data | Supabase PostgreSQL persistence and RLS. | Data survives reload; two-account isolation tests pass. |
| Responsive UX | Phone, tablet, desktop; calm visual identity; loading/error/empty states. | No major layout breakage at representative viewport sizes. |
| Delivery | GitHub, Vercel web deployment, Capacitor Android debug/release build when environment permits. | Actual status and blockers are documented; no unverified claims. |

## Explicit exclusions

- AI planning, recurring tasks, push notifications, external calendar sync, collaboration, social features, payments/subscriptions, paid APIs, and extensive theme/customization settings.
- Do not add an excluded feature simply because a library or template offers it. Record good ideas in a future-improvements section instead.

# 2. Technical architecture

## Recommended flow

**React + TypeScript + Vite** (responsive UI) → **Supabase client** → **Supabase Auth** and **PostgreSQL with RLS**. The same web app is wrapped with **Capacitor** for Android. GitHub tracks source; Vercel hosts the web build.

| Layer | Choice | Implementation guidance |
|---|---|---|
| UI | React + TypeScript + Vite | Use reusable components, typed models, and small feature-focused modules. |
| Styling | Tailwind CSS or custom CSS | Choose one approach and keep spacing, colors, typography, and states consistent. |
| Icons | Lucide React | Use icons with accessible labels; do not use icons alone where meaning may be unclear. |
| Auth/data | Supabase Auth + PostgreSQL | Use the official Supabase client and environment variables for URL and publishable/anon key. |
| Security | RLS policies | Enable RLS on every user-owned table; use auth.uid() ownership checks. |

| Layer | Choice | Implementation guidance |
|---|---|---|
| Version control | Git + GitHub | Commit in small, reviewable steps; never push without the user's approval. |
| Web hosting | Vercel free tier, subject to current limits | Set environment variables in project settings; verify deployed behavior. |
| Android | Capacitor | Android Studio/SDK and a suitable JDK may be required. Report environment blockers accurately. |

## Suggested project structure

```
src/
  app/                # app shell, routing, providers
  components/         # shared buttons, inputs, modals, states
  features/
    auth/             # sign-up, sign-in, session handling
    tasks/            # task forms, list, filters, data access
    calendar/         # month/week view and selected date
    focus/            # timer state and session saving
    progress/         # calculations, history, trends
  lib/
    supabase.ts       # configured Supabase client
    dates.ts          # shared local-date helpers
    validation.ts     # reusable validation rules
  styles/             # design tokens and global styles
  types/              # shared database/domain types
docs/
  DAILY_LOG.md
  ARCHITECTURE.md
  TEST_PLAN.md
.env.example          # placeholders only; no real secrets
.gitignore
README.md
```

This is a target structure, not a mandate to reorganize a repository that already has a sound structure. Keep existing conventions where practical.

## Environment configuration

- Use Vite client-side variables such as VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY (or the current Supabase publishable key name recommended by the installed SDK). These are public client values; RLS is the security boundary.
- Never put a Supabase service-role/secret key in React code, Vite variables, Android assets, a committed file, or a public issue. A frontend build is inspectable by users.
- Commit `.env.example` with placeholder values only. Add `.env`, `.env.local`, build output, Android local configuration, and secrets to `.gitignore` as appropriate.
- Configure production variables separately in Vercel. After changing variables, redeploy and verify the actual deployment.

## Engineering rules

- Use TypeScript types for Task and FocusSession. Avoid `any` unless a temporary, documented reason exists.
- Keep database operations in small service/helper modules rather than scattering query logic through many components.
- Represent loading, success, empty, and error states explicitly.
- Use a single, documented date convention. Database due dates are calendar dates (`date`), not UTC timestamps. Store session instants as `timestamptz`.
- Use local calendar dates for 'today' and selected calendar days. Avoid comparing date-only strings to UTC timestamps without conversion.
- Use stable IDs, not array positions, as React keys.

• Do not silently swallow failed writes. Tell the user what failed and allow retry where practical.

# 3. Database design and security

## Tasks table

| Column | Suggested type | Rules / purpose |
|---|---|---|
| id | uuid | Primary key; default gen_random_uuid(). |
| user_id | uuid | NOT NULL; references auth.users(id), preferably ON DELETE CASCADE. |
| title | text | NOT NULL; trim whitespace; enforce non-empty title and sensible maximum length, e.g. 160 characters. |
| description | text | Nullable; optional; set a practical maximum length. |
| due_date | date | Nullable; represents a calendar day without a time. |
| priority | text or enum | NOT NULL; one of `low`, `medium`, `high`; default `medium`. Add CHECK constraint or enum. |
| category | text or enum | NOT NULL; one of `study`, `work`, `personal`, `projects`; add CHECK constraint. |
| is_completed | boolean | NOT NULL DEFAULT false. |
| completed_at | timestamptz | Nullable; set when completed, clear when reopened. |
| created_at | timestamptz | NOT NULL DEFAULT now(). |
| updated_at | timestamptz | NOT NULL DEFAULT now(); update on edits via application or trigger. |

## Focus sessions table

| Column | Suggested type | Rules / purpose |
|---|---|---|
| id | uuid | Primary key; default gen_random_uuid(). |
| user_id | uuid | NOT NULL; references auth.users(id), ON DELETE CASCADE. |
| started_at | timestamptz | NOT NULL; actual start instant. |
| ended_at | timestamptz | Nullable until a session is completed; actual end instant. |
| duration_seconds | integer | NOT NULL DEFAULT 0; non-negative CHECK constraint. |
| task_id | uuid | Nullable FK to tasks(id); decide ON DELETE SET NULL so history survives task deletion. |
| status | text or enum | Use a small allowlist such as `completed`, `cancelled`; do not save unfinished timers as completed. |
| created_at | timestamptz | NOT NULL DEFAULT now(). |

Consider adding a database constraint or trigger to ensure a linked task belongs to the same user. RLS on the session table alone protects session ownership, but it does not automatically validate the ownership of every referenced task.

## Indexes and data integrity

- Add indexes for common queries, such as tasks `(user_id, due_date)`, tasks `(user_id, is_completed)`, and focus sessions `(user_id, started_at)`. Confirm with query patterns; avoid adding many unnecessary indexes.
- Use database constraints for allowed priority/category/status values and non-negative durations. Validate in the UI too, but never rely only on UI validation.

- On task completion, set `is_completed = true` and `completed_at = now()`. On reopen, set false and clear `completed_at`.
- Decide and document delete behavior: deleting a task should not accidentally delete the user's focus history. `task_id ON DELETE SET NULL` is a sensible default.
- Use migrations or saved SQL scripts in the repository so schema changes are repeatable and reviewable.

### RLS policy requirements

Enable RLS on both user-owned tables before using real accounts. A user should be allowed to select, insert, update, and delete only rows where `user_id = auth.uid()`. For inserts, require the new row's `user_id` to equal `auth.uid()`. For updates, use both `USING` and `WITH CHECK` so ownership cannot be transferred to another user.

```
Example policy shape — adapt and review before running:
ALTER TABLE public.tasks ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users manage own tasks" ON public.tasks
  FOR ALL TO authenticated
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);

ALTER TABLE public.focus_sessions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users manage own focus sessions" ON public.focus_sessions
  FOR ALL TO authenticated
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);
```

These are illustrative policy shapes, not a complete migration. Antigravity must inspect actual schema names, test policy behavior, and consider task/session relationship constraints before applying SQL.

### Two-account isolation test

1. Create test accounts A and B. Sign in as A and create at least two tasks and one focus session.
2. Sign out and sign in as B. Confirm B sees no records created by A.
3. Attempt direct client queries for A's known record IDs while signed in as B. Reads should return no protected data.
4. Attempt update and delete operations against A's IDs as B. They must not modify A's records.
5. Attempt to insert a row with `user_id` set to A while signed in as B. The database must reject it.
6. Sign in as A again and verify A's records remain intact. Record exact results and any limitations.

# 4. Behavior definitions and calculation rules

### Task state and filters

- Tasks without a due date must remain available in the general Tasks view but do not appear on a dated calendar day.
- Overdue means: incomplete, due_date is before today's local calendar date. Completed past-due tasks are not counted as currently overdue.
- 'Due today' means due_date equals today's local calendar date, whether complete or incomplete; dashboard can separate completed and incomplete due-today counts.
- Filters should combine predictably (AND semantics): e.g. category=Study plus priority=High plus status=Incomplete shows tasks meeting all selected filters.
- Deleting a task should require an intentional action and a clear confirmation if the operation is destructive. If a focus session is linked, retain session history and clear the task link.

### Focus timer rules

- Timer display should derive elapsed time from timestamps and accumulated paused duration, not count only UI interval ticks; background tabs and device sleep can throttle timers.

- Define session behavior: start begins a run; pause freezes elapsed time; resume continues; reset discards the current unsaved run after confirmation if meaningful elapsed time exists.
- Save a session only when the user completes it. A saved completed session needs accurate started_at, ended_at, duration_seconds, status, and optional task_id.
- Prevent duplicate saves from double-clicks. Show a saving state and handle database failure without falsely displaying a saved session.
- Clarify whether very short sessions are allowed. A practical Version 1 rule is to allow completed sessions of at least one second and to reject zero/negative durations.

### Progress calculations

| Metric | Recommended definition |
|---|---|
| Completion rate | Completed tasks in selected period ÷ tasks created in that period × 100. If the denominator is zero, display “No tasks in this period” rather than 0% or 100%. |
| Overdue count | Count of incomplete tasks with due_date earlier than today's local date. Define whether this metric is current snapshot or period history; use current snapshot on My Day. |
| Focus minutes | Sum duration_seconds of completed sessions whose ended_at falls inside the selected period, divided by 60. Display rounded minutes; preserve exact seconds in storage. |
| Weekly trend | Use a clearly labelled seven-day or calendar-week range consistently. Include days with zero activity as zero, not missing or fabricated values. |
| No-data period | Show a neutral empty state and explain that no qualifying activity exists. Never invent chart values. |

Make the period boundaries explicit in the UI (for example, 'This week: Mon–Sun' or 'Last 7 days'). Use one consistent definition across labels, calculations, and tests.

## 5. Seven-day development plan

Each day is a work session with a narrow goal. Antigravity must stop after the day's work, report what actually happened, and wait for explicit approval. The commit messages below are suggestions; create a commit only after the user has reviewed changes and authorized the Git action.

# Day 1 — Foundation and UI

## Objectives

- Understand the existing repository and preserve its working parts.
- Establish the responsive app shell, visual tokens, navigation, and placeholder screens.
- Set up GitHub-facing documentation and a repeatable daily log.

## Implementation steps

1. Inspect the repository: list files, read package.json, inspect existing routes/components, check `git status`, and identify the current branch. Do not overwrite user changes.
2. If no app exists, scaffold React + TypeScript + Vite. If an app exists, verify dependencies and scripts before changing anything.
3. Choose Tailwind CSS or custom CSS based on current repository state. Do not install both without a concrete reason.
4. Define design tokens: deep forest green, muted sage, warm neutral background, readable text, border color, spacing scale, and focus ring. Use the turtle motif subtly (shell geometry or restrained emblem), not cartoon illustrations everywhere.
5. Build shared app shell and navigation for My Day, Tasks, Calendar, Focus, and Progress. Use accessible links/buttons and indicate the current section.
6. Create responsive placeholder screens with useful empty-state copy. Label planned/unfinished behavior honestly; do not show fake task or focus metrics.
7. Add README with project purpose, planned stack, setup commands, environment variable names, and current status. Add `docs/DAILY_LOG.md`, `docs/ARCHITECTURE.md`, and `.env.example` with placeholders if appropriate.
8. Check `.gitignore` for `.env`, `.env.*` except `.env.example`, `node_modules`, `dist`, local Android/IDE files, and generated secrets. Inspect before editing.

## Likely files

- `package.json`, `src/main.tsx`, `src/App.tsx` or routing equivalent
- `src/components/AppShell.tsx`, `src/components/Sidebar.tsx` or mobile navigation equivalent
- `src/styles/*` or `src/index.css`, `tailwind.config.*` if Tailwind is selected
- `README.md`, `docs/DAILY_LOG.md`, `docs/ARCHITECTURE.md`, `.env.example`, `.gitignore`

## Testing checklist

- Run the actual available install/build/lint/typecheck commands; do not assume scripts exist.
- Open each of the five screens and verify navigation works.
- Test at representative phone (around 375 px), tablet (around 768 px), and desktop (around 1280 px) widths.
- Tab through navigation and controls; confirm visible focus and no inaccessible icon-only actions.
- Confirm the UI contains no fabricated productivity numbers and no console errors caused by the new shell.

## Definition of completion

A working Vite development server and production build (if scripts support them); five navigable responsive screens; consistent Kachuno tokens; basic documentation; clean or understood Git status. Do not proceed if the project cannot run without documenting the blocker.

## Suggested commit

```
chore: set up Kachuno app shell and responsive design
```

## Common problems and troubleshooting

- Dependency install fails: inspect Node/npm versions and package manager lockfile; use the repository's existing package manager rather than creating a second lockfile.
- Blank screen: inspect terminal and browser console; check entry point imports and route fallback.
- Mobile navigation overflows: use a compact menu or bottom navigation; avoid shrinking desktop navigation into unreadable text.
- Git shows unrelated modifications: do not reset or discard them. Ask the user what to do and only stage intended files.

# Day 2 — Task management

## Objectives

- Implement reliable task CRUD and status transitions.
- Add filters and derive My Day from real task data.
- Validate input and cover edge cases.

## Implementation steps

1. Define a typed Task model matching the agreed database fields, plus explicit category and priority unions.
2. Create a task data-access layer with a temporary local in-memory adapter or local-only persistence for this day if Supabase is not connected yet. Clearly label temporary storage and keep the interface easy to replace.
3. Build a task form for title, optional description, due date, priority, and category. Trim title, reject blank titles, and set sensible character limits.
4. Implement create, edit, complete, reopen, and delete. Keep completed_at behavior consistent in the domain logic, even if cloud persistence comes later.
5. Implement filters for due date/range, category, priority, and status. Combine filters predictably and offer a clear-filters action.
6. Build task list/card rows with priority, due date, completion state, edit action, and delete action. Make the completion control keyboard accessible.
7. Build My Day from actual task records: due today, overdue, completed/incomplete counts, and today's focus minutes placeholder should remain clearly unavailable until focus sessions exist.
8. Handle no tasks, no filter matches, invalid forms, loading, and write failures with useful messages.

## Likely files

- `src/features/tasks/types.ts`, `taskService.ts`, `TaskForm.tsx`, `TaskList.tsx`, `TaskFilters.tsx`
- `src/features/my-day/MyDayPage.tsx`, `src/lib/dates.ts`, shared empty/error components

## Testing checklist

- Create task with only a title; create task with all fields; reject whitespace-only title.
- Edit every field; complete then reopen; confirm completion timestamp logic.
- Delete a task and confirm it disappears from active lists; test canceling the confirmation.
- Combine category, priority, status, and date filters; clear filters and verify the full list returns.
- Test no due date, due today, yesterday, future date, completed overdue task, and incomplete overdue task.
- Reload behavior should match the current storage adapter's documented behavior; do not claim cloud persistence yet.

## Definition of completion

All task operations work against the chosen day-2 data layer; filters produce expected results; My Day is derived from actual records; form validation and empty/error states exist; the build and typecheck pass if configured.

## Suggested commit

```
feat: add task management and filters
```

## Common problems and troubleshooting

- Date shifts by one day: treat due dates as date-only strings and avoid converting `YYYY-MM-DD` to a UTC timestamp for display.

- Filtered task disappears unexpectedly: inspect combined filter predicates and make filter semantics visible.
- Completion state differs between screens: centralize task updates and use a single source of truth.
- Temporary storage is mistaken for cloud storage: label it in documentation and replace it on Day 4; never claim cross-device persistence.

# Day 3 — Calendar and focus timer

## Objectives

- Add monthly/weekly calendar views linked to task due dates.
- Implement an accurate timer state machine.
- Save completed focus sessions through a service boundary.

## Implementation steps

1. Create shared date helpers for local date formatting, start-of-week, month boundaries, and day comparison. Decide whether the week starts Monday and document it.
2. Build month and week views with previous/next navigation and a clear selected-date state. Ensure date cells have accessible labels and keyboard operation.
3. Render tasks on their due dates; tasks without due dates remain outside the calendar. Selecting a date shows that day's tasks and allows opening/editing them.
4. Define timer states: idle, running, paused, saving, and error. Keep state transitions explicit to avoid impossible combinations.
5. Use timestamps and accumulated paused duration to calculate elapsed seconds. A display interval may refresh the UI, but must not be the source of elapsed-time truth.
6. Support start, pause, resume, and reset. Define reset behavior for an active session and prevent accidental loss where appropriate.
7. On complete, capture started_at, ended_at, duration_seconds, optional task_id, and completed status. Prevent duplicate submissions and show saved/failed feedback.
8. Create a session history list or at minimum the data-access and UI foundation required for Day 5. Ensure the UI never says a session was saved before the write succeeds.

## Likely files

- `src/features/calendar/CalendarPage.tsx`, `CalendarGrid.tsx`, `CalendarToolbar.tsx`
- `src/features/focus/FocusPage.tsx`, `useFocusTimer.ts`, `focusSessionService.ts`
- `src/lib/dates.ts`, `src/types/focus.ts`

## Testing checklist

- Calendar handles 28/29/30/31-day months, year boundaries, and months beginning on different weekdays.
- Task appears on its exact due date and selected-date list matches the grid.
- Timer can start, pause, resume, complete, and reset. Paused time does not count as focus time.
- Simulate delayed tab/background behavior; elapsed time remains consistent with timestamps.
- Completed session stores a plausible duration and correct timestamps; linked task is optional.
- Double-clicking complete does not create duplicate sessions; failed save is visible and retryable.

## Definition of completion

Month/week navigation and selected-date filtering work; timer transitions behave correctly; completed sessions are represented by a typed model and save path; edge cases are tested or recorded as blockers.

## Suggested commit

```
feat: add calendar and focus timer
```

## Common problems and troubleshooting

- Timer loses time after tab is hidden: derive elapsed time from timestamps, not interval tick count.
- Timer counts paused time: store accumulated elapsed time at pause and reset the active segment on resume.
- Calendar shows off-by-one dates: audit date-only parsing and local timezone conversions.
- Focus session appears twice: disable completion while saving and guard the save operation against repeat invocation.

# Day 4 — Supabase and authentication

## Objectives

- Connect Supabase Auth and PostgreSQL.
- Create repeatable schema and RLS policies.
- Replace temporary task/session persistence and verify account isolation.

## Implementation steps

1. Create or inspect the Supabase project using the user's account. Do not request or store the service-role key in frontend files.
2. Add environment variables locally and `.env.example` placeholders. Verify `.env` is ignored by Git before staging.
3. Create versioned SQL/migration files for tasks, focus_sessions, constraints, indexes, updated_at behavior, and RLS policies. Review SQL before running it.
4. Implement Supabase client initialization with clear handling when required environment values are missing.
5. Build sign-up, sign-in, sign-out, and auth loading/session restoration. Use clear validation and error messages without exposing unnecessary backend details.
6. Protect signed-in app routes and provide a sign-in screen when there is no authenticated session. Handle expired sessions.
7. Replace temporary storage adapter with Supabase-backed methods while preserving the UI/service contract. Include user ownership in inserts and rely on RLS for enforcement.
8. Add the two-account isolation test from Section 3. Verify reads, inserts, updates, and deletes. Test linked task/session ownership.
9. Update README with safe setup steps, environment variable names, and SQL migration instructions. Never include real keys.

## Likely files

- `src/lib/supabase.ts`, `src/features/auth/AuthProvider.tsx`, `SignInPage.tsx`, `SignUpPage.tsx`
- `src/features/tasks/taskService.ts`, `src/features/focus/focusSessionService.ts`
- `supabase/migrations/*.sql` or `supabase/schema.sql`, `.env.example`, `README.md`

## Testing checklist

- Missing environment values produce an understandable setup message, not a blank screen.
- Sign-up, sign-in, session restore, sign-out, and invalid credentials behave as expected.
- Task/session writes survive reload and sign-in from the same account.
- Account B cannot read, modify, delete, or create rows owned by account A.
- RLS is enabled on every user-owned table; no frontend service-role key exists.
- Search Git diff and tracked files for likely secrets before any commit or push.

## Definition of completion

Auth and cloud persistence work in the configured environment; migrations and RLS policies are saved in the repository; two-account tests pass or exact blockers are documented; secret handling is reviewed.

## Suggested commit

```
feat: connect Supabase auth and persistence
```

## Common problems and troubleshooting

- RLS returns no rows: verify authenticated session, policy, and row user_id; do not disable RLS as a workaround.
- Insert rejected: inspect required fields, enum/check values, FK references, and user_id assignment.
- Auth redirect loops: check session loading state before route decisions and confirm redirect URLs in Supabase settings.
- Environment variable changes do not appear: restart Vite locally; redeploy after updating Vercel settings.
- Never paste service-role secrets into Antigravity chat, source code, GitHub, or issue reports.

# Day 5 — Progress and history

## Objectives

- Build task and focus history views.
- Implement transparent metrics and weekly trends.
- Test edge cases and empty periods.

## Implementation steps

1. Define the period selector/range and make it explicit in the UI. Keep a single consistent definition for all charts and labels.
2. Create task history using completed_at for completed tasks and created_at/due_date where relevant. Distinguish completed-in-period from currently completed.
3. Create focus history from saved completed sessions. Show date/time, duration, and linked task when available.
4. Implement completion rate using the agreed denominator: completed tasks created in the selected period divided by all tasks created in that period. If denominator is zero, show a no-data message.
5. Calculate focus minutes from completed session durations whose ended_at falls inside the selected period. Include zero-activity days in weekly trends.
6. Show current overdue count separately from historical period metrics so the meanings do not conflict.
7. Use accessible, simple visualizations or compact bars. Include text labels and do not rely on color alone.
8. Test calculations with a fixed set of sample records, then verify against database queries or hand-calculated expected results.

## Likely files

- `src/features/progress/ProgressPage.tsx`, `TaskHistory.tsx`, `FocusHistory.tsx`, `WeeklyTrend.tsx`
- `src/features/progress/metrics.ts`, `src/features/progress/metrics.test.ts`

## Testing checklist

- No tasks in period: no divide-by-zero; show 'No tasks in this period'.
- One task, incomplete: 0% if denominator is one; one task completed: 100%.
- Completed task created outside the period: verify it is excluded under the chosen formula.
- Focus sessions at period boundaries: test exact start/end inclusion rules and timezone handling.
- Zero activity on a weekday is displayed as zero; it is not omitted or replaced with invented data.
- Overdue count excludes completed tasks and tasks due today.

## Definition of completion

History and progress use saved data; formulas are documented; sample calculations pass; empty and no-data periods are understandable; weekly trends are honest and consistently labelled.

## Suggested commit

```
feat: add progress dashboard and history
```

## Common problems and troubleshooting

- Completion rate looks surprisingly low/high: check denominator, date boundaries, and whether created_at or completed_at defines the period.

- Totals differ across screens: share metric helpers and common date-range logic.
- Graph is empty despite data: inspect timezone bounds and session status filtering.
- Percentages show NaN: explicitly handle zero denominator before division.

# Day 6 — Responsive polish and Android

## Objectives

- Improve accessibility, responsive behavior, validation, and error states.
- Configure Capacitor and attempt a debug Android build if the environment permits.
- Fix defects without expanding scope.

## Implementation steps

1. Audit every screen at phone, tablet, and desktop sizes. Check navigation, forms, modals, calendar, history tables/cards, and timer controls.
2. Run keyboard-only navigation. Verify focus indicators, semantic headings, form labels, button names, color contrast, and screen-reader-friendly status/error messages.
3. Review loading skeletons/spinners, empty states, network failures, form errors, and session-expired handling.
4. Check dependency list for unnecessary packages and remove only those proven unused and safe to remove.
5. Install/configure Capacitor using current official project instructions and compatible package versions. Set a stable app ID such as a reverse-domain identifier controlled by the developer (example only: `com.example.kachuno`; replace before release).
6. Configure app name, web asset directory, and Android platform. Generate/sync Android project only after the web app builds.
7. Build and run a debug APK on an emulator/device if Android Studio, SDK, JDK, and required platform tools are installed.
8. Test login, task CRUD, calendar, timer, progress, keyboard/mobile interactions, and safe area/keyboard behavior on Android where possible.
9. Document any native-platform blockers instead of claiming Android works based only on a successful web build.

## Likely files

- `capacitor.config.ts`, `android/` generated project, `package.json`
- `src/styles/*`, shared form/error components, app metadata/configuration

## Testing checklist

- Responsive checks at representative narrow, medium, and wide widths; no horizontal scrolling except where intentionally needed.
- Tap targets are comfortable; dialogs fit the viewport; the on-screen keyboard does not hide important form controls.
- Focus timer display and state remain usable when app/tab is backgrounded; do not promise background execution beyond implementation.
- Run configured lint, typecheck, unit tests, and production build.
- Android debug build succeeds, installs, opens, and reaches core screens—or the exact failure and environment limitation are recorded.

## Definition of completion

The main UI is responsive and accessible enough for a first release; known bugs are fixed or listed; Capacitor configuration is committed; Android debug build status is verified and documented.

## Suggested commit

```
feat: package Android app and improve responsive UX
```

## Common problems and troubleshooting

- Capacitor sync fails: verify web build output directory, compatible package versions, and configuration; read the first meaningful error.
- Android SDK missing: document the requirement and provide setup instructions; do not substitute a claim that the APK was built.
- Keyboard covers fields: adjust scroll behavior and focus management; test on a real emulator/device.
- UI differs from browser: test platform-specific viewport, safe area, and status bar behavior; avoid broad rewrites.

# Day 7 — Deployment and final verification

## Objectives

- Run final checks and deploy the web app.
- Create an Android release artifact if possible.
- Finish documentation, acceptance evidence, and publishing notes.

## Implementation steps

1. Review the full diff, Git status, TODO markers, known issues, and daily logs. Confirm no unrelated files or secrets are staged.
2. Run the actual scripts available in package.json: lint, typecheck, tests, and production build. Record exact commands, exit status, and key output.
3. Deploy to Vercel using the repository integration or documented CLI flow. Configure only required client environment values in deployment settings.
4. Verify the deployed URL in a fresh browser session: sign-up/sign-in, CRUD, date correctness, focus-session save, progress, responsive UI, and error handling.
5. Check Supabase allowed redirect URLs and production configuration. Confirm RLS is enabled and two-account isolation was tested against the deployed environment where practical.
6. Create an Android release build only if signing/build setup is available. Distinguish a debug APK, signed release APK, and Android App Bundle (AAB); they are not interchangeable.
7. Complete README setup/deployment instructions, architecture, test plan, known issues, and `docs/DAILY_LOG.md`.
8. Document remaining Google Play tasks, including developer account registration fee, app identity, store listing, privacy policy/data safety disclosures, content rating, screenshots, signing, target API requirements, and current testing/review requirements.
9. Review final acceptance checklist and produce a truthful status summary. Do not mark an item complete without evidence.
10. Commit and push only after the user reviews the proposed files and explicitly authorizes the Git action.

## Likely files

- `README.md`, `docs/DAILY_LOG.md`, `docs/ARCHITECTURE.md`, `docs/TEST_PLAN.md`
- Vercel project settings (not committed secrets), `capacitor.config.ts`, Android release configuration

## Testing checklist

- Production build passes in the actual environment.
- Deployed site works in a clean browser session, not only on localhost.
- Production environment variables are configured; no secret keys are exposed in built assets or Git history.
- Core flows and RLS are verified; any untested security scenario is clearly labelled unverified.
- Android build type, file artifact, installation status, and test device/emulator are recorded.
- README and logs match the actual repository state and command results.

## Definition of completion

The web deployment has a verified URL and documented test evidence; Android build status is precise; all acceptance items are marked Pass, Fail, or Not verified; documentation is complete; no unapproved Git push occurred.

## Suggested commit

```
docs: finalize deployment and project documentation
```

## Common problems and troubleshooting

- Deployment build fails but local build passes: compare Node version, lockfile, environment variables, build command, and output directory.
- Auth works locally but not in production: inspect Supabase redirect allowlist and deployed origin configuration.
- Android release signing is incomplete: keep the build status as blocked and document the next steps; do not commit signing credentials.
- Google Play review exceeds the schedule: this is expected to be outside the developer's control. Report app readiness separately from store approval.

# 6. Daily GitHub workflow

Git operations are user-controlled. Antigravity may propose commands and inspect status, but must not commit or push unless the user explicitly authorizes that action for the current session.

## Safe end-of-session sequence

1. Run `git status --short --branch` and inspect the current branch and untracked files.
2. Run `git diff` and, if useful, `git diff --staged`. Review every changed file for accidental edits and secrets.
3. Run relevant checks before staging. Record the exact commands and results in `docs/DAILY_LOG.md`.
4. Stage only intended files by explicit path, e.g. `git add src/features/tasks docs/DAILY_LOG.md`; avoid `git add .` when unrelated files may exist.
5. Inspect `git diff --staged` one final time.
6. Show the proposed commit message and summary. Wait for explicit approval before `git commit`.
7. After an authorized commit, record the hash using `git rev-parse --short HEAD` and verify `git status`.
8. Wait for explicit approval before `git push`. Then verify push output and remote branch status.
9. Update the daily log with blockers and the next day's smallest useful starting point.

## Repository hygiene checklist

- Never commit `.env`, `.env.local`, service-role keys, access tokens, signing keystores, passwords, real user data, or database dumps.
- Use `.env.example` with fake placeholder values. Inspect tracked files and history if a secret was accidentally committed; removing the line in a later commit is not enough. Rotate exposed credentials immediately.
- Do not run destructive commands (`git reset --hard`, force push, broad deletion) without explicit permission and a clear explanation.
- Do not overwrite or discard pre-existing user changes. If the working tree is already dirty, report it before editing.
- Use one focused commit per completed day when possible. If work spans multiple sessions, use truthful smaller commits only with approval.

## Suggested command reference

```
git status --short --branch
git diff
git diff --staged
git add <specific-paths>
git diff --staged
git commit -m "feat: descriptive change"
git rev-parse --short HEAD
git status --short --branch
git push
```

The command list is a reference, not permission to execute every command. The user controls commit and push authorization.

# 7. Antigravity master prompt

Copy the following prompt into Antigravity after uploading this PDF to the repository workspace. Use it again at the start of each day, changing the approved day number.

You are the implementation assistant for Kachuno, a beginner-built productivity app. The uploaded document “Kachuno — 7-Day Project Blueprint” is the project specification. Follow it as the source of truth unless the owner explicitly approves a change.

**WORKING RULES**

1. Read the complete blueprint first. Then inspect the repository, package.json, current branch, git status, relevant source files, and existing documentation before editing.

2. Work on ONLY the one day the owner has approved. If no day is specified, do not edit; ask which day is approved.

3. Before implementation, provide a short plan: goal, files likely to change, commands/tests to run, risks, and any required user decisions. Wait for approval if the plan would alter architecture, delete/replace existing code, change scope, or require an external account/paid action.

4. Preserve working code and all pre-existing user changes. Prefer small, focused edits. Do not rewrite the application or reorganize files without a clear reason.

5. Stay within Version 1 scope. Do not add excluded features. Record future ideas in documentation only.

6. Explain unfamiliar decisions in beginner-friendly language. Do not hide important trade-offs.

7. Use TypeScript, validate inputs, handle loading/error/empty states, and keep UI accessible and responsive.

8. Treat database security as essential. Enable and test RLS on user-owned tables. Never expose a Supabase service-role/secret key in frontend code, Vite environment variables, Android assets, logs, screenshots, or GitHub. Never disable RLS to make a feature appear to work.

9. Do not commit, push, publish, deploy, register an account, spend money, change account settings, or perform destructive Git actions unless the owner explicitly authorizes that specific action. You may prepare commands and explain them.

10. Do not claim any test, typecheck, build, security review, deployment, Android build, or RLS test succeeded unless you actually ran it and inspected the result. Report exact commands, exit status, and relevant output. If a command could not run, say why.

11. Do not invent metrics or demo data that looks real. Do not use guilt-based copy. Handle empty periods without misleading percentages.

12. If blocked by credentials, missing tools, SDKs, permissions, or environment setup, stop the affected part, explain the blocker, and offer the smallest next step. Do not work around security boundaries.

13. At the end of the approved day, STOP. Do not begin the next day until the owner approves it.

**REQUIRED DAY-END REPORT**

- Day and objective
- Summary of behavior implemented
- Files created/modified and why
- Commands actually run and exact results
- Manual tests performed and outcomes
- Security/privacy checks performed and limitations
- Known bugs, blockers, and unverified items
- Git status and proposed commit message
- Daily log updated (yes/no)
- Next step for the next day
- Explicit statement that you are waiting for approval

**GIT RULE**

Show `git status` and review the diff. Stage only intended files. Propose the commit message. Wait for explicit authorization before committing. Wait for separate explicit authorization before pushing. Never include secrets or private data.

Start by inspecting the repository and summarizing its current state. Do not implement until the owner approves the plan for the selected day.

# 8. Deployment and publishing considerations

## Web deployment

- Create a GitHub repository and connect it to Vercel using the current setup flow. Confirm the framework preset, build command, and output directory match the actual project.
- Add only the public Supabase project URL and publishable/anon client key to the Vercel environment. Never add a service-role key.
- Set Supabase Auth site URL and allowed redirect URLs for localhost and the deployed domain as appropriate.
- Deploy a preview first if available. Test the real URL in a clean browser session and on a mobile-sized viewport.

- Check current free-tier limits, bandwidth/build quotas, and acceptable-use terms before treating the service as a permanent zero-cost host.

## Android build and Google Play

- Capacitor wraps the web app; it does not automatically guarantee native behavior, store approval, offline data access, or background timer execution.
- Android development may require Android Studio, Android SDK components, and a compatible JDK. The build environment may not have these installed.
- A debug APK is for development/testing. A Play release commonly requires a properly signed Android App Bundle (AAB), subject to current Google Play requirements.
- Google Play developer registration has an unavoidable fee and verification requirements; confirm current amounts and rules on the official Play Console before budgeting. Approval timing is not guaranteed.
- Prepare an accurate store listing, app icon, screenshots, privacy policy, Data safety declarations, content rating, target API compliance, app signing, and any required testing track or tester period.
- Because Kachuno stores account-linked task and focus data, explain data collection, use, retention, deletion, and third-party processing truthfully. Publish a privacy policy that matches the actual implementation.

## Zero-cost strategy

| Likely free during development | Potential unavoidable or changing costs |
|---|---|
| VS Code/Antigravity access already available; Git; GitHub; React/Vite; Lucide; Supabase free tier; Vercel free tier; Android Studio/SDK. | Google Play developer registration; optional domain; usage beyond free-tier limits; paid AI/IDE quota; any future paid hosting or compliance service. |

Do not guarantee a ■0 lifetime operating cost. Free tiers have limits and can change; production usage, custom domains, store registration, or usage spikes may introduce costs.

# 9. Final acceptance checklist

Mark each item **Pass**, **Fail**, or **Not verified**. Add evidence for every Pass. A screenshot alone is not enough for a security claim; record the test method and result.

| Acceptance item | Evidence required | Status |
|---|---|---|
| Authentication works | Sign-up, sign-in, session restoration, and sign-out verified. | ■ Pass ■ Fail ■ Not verified |
| Task operations work | Create, edit, complete, reopen, delete, due date, priority, category, and filters tested. | ■ Pass ■ Fail ■ Not verified |
| Calendar dates match due dates | Test month/week views, selected date, timezone boundaries, and tasks without due dates. | ■ Pass ■ Fail ■ Not verified |
| Focus sessions save accurately | Pause/resume timing, completion, duration, optional task link, duplicate-save prevention, and persistence tested. | ■ Pass ■ Fail ■ Not verified |
| Progress calculations are correct | Hand-calculated sample cases, period boundaries, zero denominator, and zero-activity days tested. | ■ Pass ■ Fail ■ Not verified |
| RLS prevents cross-account access | Two accounts tested for select/insert/update/delete; ownership tampering rejected. | ■ Pass ■ Fail ■ Not verified |
| Mobile layout is usable | Representative phone/tablet/desktop widths checked; keyboard, forms, and dialogs usable. | ■ Pass ■ Fail ■ Not verified |
| Web deployment is verified | Real deployed URL tested in a clean session; environment and auth redirects work. | ■ Pass ■ Fail ■ Not verified |

| Acceptance item | Evidence required | Status |
|---|---|---|
| Android build status is documented | Record build type, command, result, artifact location, and device/emulator test; or precise blocker. | ■ Pass ■ Fail ■ Not verified |
| GitHub contains no secrets | Tracked-file review and diff scan completed; no service-role key, `.env`, tokens, or signing credentials. | ■ Pass ■ Fail ■ Not verified |
| Documentation and daily logs complete | README, setup, architecture, test plan, known issues, and seven daily log entries/statuses updated. | ■ Pass ■ Fail ■ Not verified |

# 10. Daily progress log template

Create and maintain `docs/DAILY_LOG.md`. Keep entries factual and concise. Use the template below for each session; do not mark work complete based only on intention.

```
# Kachuno Daily Development Log

## Day N — YYYY-MM-DD

**Approved scope:**

**Goal:**

### Work completed

-

### Files changed

- `path/to/file` — reason for change

### Verification

| Command or test | Actual result | Pass / Fail / Blocked |

|---|---|---|

| `npm run build` | | |

| `npm run lint` (if configured) | | |

| `npm run typecheck` (if configured) | | |

| Manual test | | |

### Security and data checks

- RLS / ownership checks:

- Secrets scan:

- Limitations:

### Git

- Branch:

- Commit message proposed:

- Commit hash (only after authorized commit):

- Push status (only after authorized push):

- Working tree status:

### Blockers / known issues

-

### Next smallest step

-
```

### Owner approval

- Current day reviewed: yes / no
- Next day approved: yes / no

# 11. README starter checklist

- One-paragraph project overview, tagline, screenshots only when they show the real app, and current development status.
- Tech stack and prerequisites (Node version, package manager, Git, optional Supabase account, optional Android Studio/SDK/JDK).
- Clone/install/run commands copied from the actual repository scripts; do not guess script names.
- Environment variable names and how to obtain values safely; no real secrets.
- Database migration/RLS setup instructions and how to create test users.
- Test commands, known issues, deployment instructions, and Android build status.
- Contributing or project workflow note explaining day-by-day scope and review-before-commit approach.

# 12. Risks and decisions to resolve

| Risk / decision | Recommended handling |
|---|---|
| Unknown existing repo state | Inspect before edits; preserve existing changes; ask before destructive or architectural changes. |
| Free-tier policy changes | Verify current limits before launch and document any constraint that affects expected use. |
| Timezone/date-only bugs | Keep due_date as a date-only value; centralize local date calculations; test boundaries. |
| Timer interruptions | Use timestamps for elapsed time; explicitly define behavior on reload/app suspension. |
| RLS misconfiguration | Write policies as migrations; test with two users; do not rely on UI filtering. |
| Account recovery | Decide whether password reset is in initial auth scope; if not implemented, document limitation rather than imply recovery exists. |
| Data deletion/export | Version 1 should at minimum document how an account/data deletion request will be handled; consider an in-app deletion flow only if approved as necessary scope. |
| Google Play policy readiness | Check current requirements close to release; don't promise store approval by Day 7. |
| Commit/push authority | Require explicit approval for each commit/push action; never infer permission from the daily plan. |

# Final instruction

Kachuno is successful when its core actions are reliable, its data is private, and its progress metrics are honest. Build the smallest complete version first. At the end of every day, report evidence—not optimism—and wait for approval before moving on.